Universität Augsburg

Fakultät für Angewandte Informatik

Bachelorarbeit

Entwurf und Evaluierung verschiedener Regelkonzepte für einen Quadrocopter in der Lehre

---

vorgelegt von:	Maximilian Schweiger
eingereicht am:	20.03.2026
Studiengang:	Ingenieurinformatik
Anfertigung am Lehrstuhl:	Regelungstechnik
	Fakultät für Angewandte Informatik
1. Gutachter:	Prof. Dr.-Ing. habil. Christoph Ament
2. Gutachter:	Prof. Dr.-Ing. Lars Mikelsons
Wissenschaftliche Betreuer:	M.Sc. Ferdinand Friedrich
	M.Sc. Johannes Wüstner

Inhaltsverzeichnis

1. Einleitung 1
1.1. Motivation 1
1.2. Zielsetzung und Aufbau 2
2. Systembeschreibung 4
2.1. Hardwareaufbau des Quadrocopters 5
2.2. Softwarearchitektur 7
2.2.1. Bodenstation Software 7
2.2.2. Onboard-Software 8
3. Modellbildung 10
3.1. Parameteridentifikation 15
3.2. Linearisiertes Modell 18
4. Reglerentwurf 21
4.1. Steuerungszuweisung 23
4.2. Lageregelung 24
4.2.1. PID 25
4.2.2. LQR mit integraler Ausgangsrückführung 27
4.2.3. Evaluierung 29
4.3. Positionsregelung 32
4.3.1. PID 32
4.3.2. LQR mit integraler Ausgangsrückführung 32
4.3.3. Evaluierung 32
4.4. Implementierung 32
5. Fazit und Ausblick 33
Literaturverzeichnis 34

1. Einleitung

Die Entwicklung autonomer Quadrocopter ist ein hochgradig interdisziplinäres Forschungsfeld an der Schnittstelle von Informatik, Physik und Regelungstechnik. Über die Forschung hinaus bieten diese Systeme eine vielseitige Plattform für die akademische Lehre, da sie komplexe theoretische Konzepte der Regelungstechnik greifbar und praxisnah vermitteln. Das vorliegende Einleitungskapitel beleuchtet zunächst die praktische Relevanz von Quadrocoptern und arbeitet deren Vorzüge als modernes Lehrmittel heraus. Darauf aufbauend wird die Zielsetzung dieser Arbeit definiert und der strukturelle Aufbau der Untersuchung skizziert.

1.1. Motivation

Aktueller Forschungsstand

Obwohl das Konzept der Multicopter bereits in der Frühzeit der Luftfahrt erprobt wurde, verhalfen erst die Fortschritte in der Mikroelektronik und Akkutechnologie der letzten drei Jahrzehnte den Quadrocoptern zum Durchbruch. Heute hat sich die Technologie zu einem vielseitig einsetzbaren Werkzeug entwickelt und ist Teil zahlreicher Forschungsprojekte. [1]

Die Grundlagen der Modellbildung und Regelungstechnik für Quadrocopter besitzen bereits ein etabliertes Theoriefundament. Deshalb konzentriert sich die aktuelle Forschung vor allem auf die Behandlung von Unsicherheiten. Hierfür werden zum Beispiel statistische Methoden in der Modellbildung oder adaptive Regelungsstrategien untersucht. In jüngerer Zeit liegt der Fokus zunehmend auf hybriden Ansätzen, die klassische Regelungsstrategien mit Methoden des maschinellen Lernens kombinieren. Dadurch sollen beispielsweise Herausforderungen in der

Schwarmintelligenz oder der Energieeffizienz adressiert werden. Die praktischen Einsatzgebiete sind dabei vielfältig und reichen von der autonomen Inspektion industrieller Anlagen über die topografische Vermessung bis hin zu logistischen Anwendungen. [2]

Quadrocopter als didaktische Plattform

Zudem eignen sich Quadrocopter hervorragend als Lehrmittel. Während die mechanische Struktur vergleichsweise einfach und wartungsarm ist, bietet das dynamische Verhalten eine Fülle an regelungstechnischen Herausforderungen. Sie weisen eine nichtlineare Dynamik mit vielfältigen Kopplungen zwischen den Bewegungsachsen auf. Da das System trotz seiner sechs Freiheitsgrade nur über vier Stellgrößen verfügt, handelt es sich um ein unteraktioniertes System, bei dem die Translation in x- und y-Richtung nur durch die Neigung des Gesamtsystems erreicht werden kann. Vielfältige aerodynamische Effekte und unsichere Umgebungen erschweren die Modellbildung und erfordern die Verwendung robuster Regelungsansätze. Trotz der damit offensichtlichen Eignung für die Lehre von fortgeschrittenen Regelungskonzepten sind Quadrocopter auch für die Vermittlung von Grundlagen der Regelungstechnik attraktiv, da sie sich in der Hover-Position annäherend wie ein entkoppeltes, lineares System verhalten.

1.2. Zielsetzung und Aufbau

Das Ziel dieser Arbeit ist die Konzeption und Realisierung einer funktionalen Basis für den Einsatz eines Quadrocopters in der Lehre. Um dieses Ziel zu erreichen, soll zunächst die Hardware konzipiert und in Betrieb genommen sowie softwareseitige Voraussetzungen für die Reglersynthese geschaffen werden. Anschließend sollen verschiedene Regelungsansätze implementiert und experimentell evaluiert werden, um die Eignung der Plattform für die Lehre zu demonstrieren.

Die Arbeit ist in vier aufeinander aufbauende Abschnitte unterteilt. Im zweiten Kapitel wird die Systembeschreibung gegeben, in der die mechanische Konstruktion, die Sensorik und Aktorik sowie die Softwarearchitektur des Quadrocopters vorgestellt werden. Das dritte Kapitel widmet sich der Modellierung, wobei sowohl

physikalische als auch datengetriebene Ansätze zur Beschreibung der Dynamik des Systems untersucht werden. Im vierten Kapitel erfolgt die eigentliche Regelungsentwicklung, bei der LQR und PID-Regler für die Lage- und Positionsregelung implementiert und anhand von realen Flugtests evaluiert werden. Im abschließenden fünften Kapitel werden die Ergebnisse zusammengefasst, die Eignung der Plattform für die Lehre untersucht und ein Ausblick auf mögliche Erweiterungen und Verbesserungen gegeben.

2. Systembeschreibung

Das Gesamtsystem gliedert sich in vier Subsysteme (vgl. Abb. 2.1):

1. Lasertracker: Erfasst die Position des Quadrocopters und übermittelt die Positionsdaten via Ethernet an die Bodenstation.
2. Bodenstation: Verarbeitet die Trackerdaten und sendet diese zusammen mit Steuerbefehlen an den Quadrocopter. Zudem wertet diese empfangene Telemetriedaten aus und kümmert sich um die Datenhaltung. Die Kommunikation mit dem Quadrocopter erfolgt kabelgebunden über eine serielle Schnittstelle.
3. Quadrocopter: Das Herzstück des Systems, welches regelungstechnisch untersucht wird.
4. Fernsteuerung (optional): Ermöglicht eine manuelle Steuerung über einen separaten Funklink.

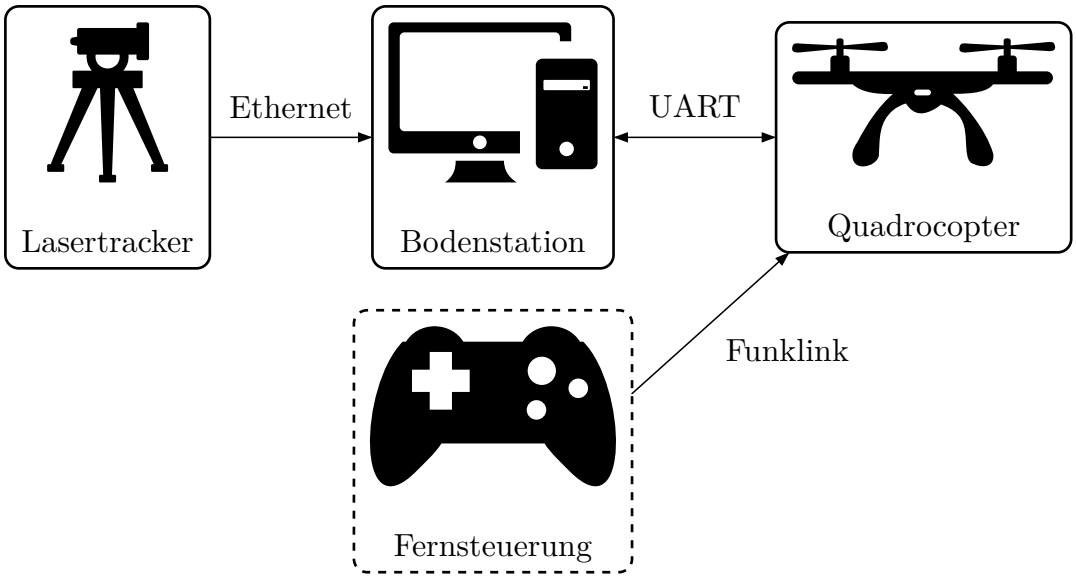


Abb. 2.1: Systemüberblick (Fernsteuerung ist optional)

Im Rahmen dieser Arbeit wurde die Avionik und Onboard-Software des Quadrocopters sowie die Software der Bodenstation entwickelt. Auf diese Komponenten soll im Folgenden genauer eingegangen werden.

2.1. Hardwareaufbau des Quadrocopters

Der Quadrocopteraufbau basiert mechanisch auf dem Quadrocopter Control System (QCS) von Emqopter [3], welches ebenfalls für die Lehre entwickelt wurde. Da aber wesentliche Teile ihrer Software proprietär sind, ist eine vollständige Integration eigener Regelungsalgorithmen nur schwer möglich. Daher wurde zwar der mechanische Aufbau inklusive des 3-DOF-Teststands übernommen (vgl. Abb. 2.2) und die meisten elektronischen Komponenten wiederverwendet, der Flugcomputer jedoch basierend auf einem ESP-WROOM-32 grundlegend neu entwickelt.t

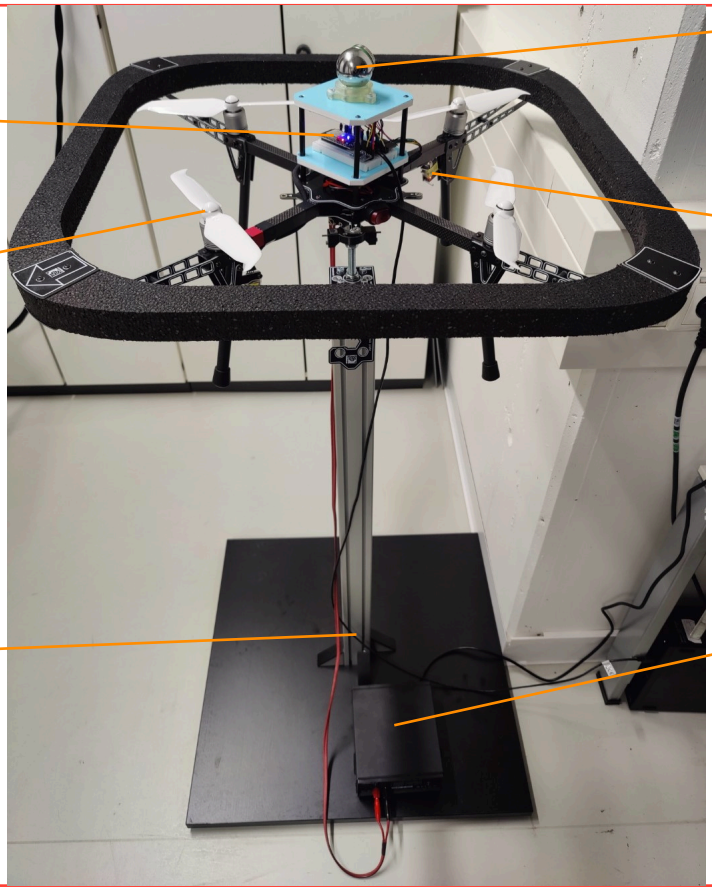


Abb. 2.2: Quadrocopter auf dem 3-DOF-Teststand

Die Avionik (Abb. 2.3) umfasst neben dem Mikrocontroller folgende Komponenten:

1. MPU6050-Sensor: Dieser dient der Erfassung der Lagedaten. Dabei wird der integrierte Digital Motion Processor (DMP) genutzt, welcher die Rohdaten des Beschleunigungssensors und Gyroskops direkt onboard fusioniert. Er liefert über die I2C-Schnittstelle gefilterte Euler-Winkel zur Lagebeschreibung.
2. RC Empfänger: Er empfängt die Steuerbefehle der Fernsteuerung und übermittelt diese via UART an den ESP32.
3. Diatone Mamba F55_128K ESC [4]: Er ermöglicht die Ansteuerung der vier Rotoren über verschiedene Protokolle. Im vorliegenden System erfolgt die Vorgabe der Rotordrehzahl über ein PWM-Signal (490 Hz). Dabei werden Pulsweiten von $1-2$ ms linear auf eine Drosselstellung von 0% bis 100% abgebildet. Zusätzlich ist an diesen noch eine externe Stromquelle angeschlossen, welche den Strom für die Rotoren liefert und über die zukünftig auch der Mikrocontroller angeschlossen werden kann.
4. BLDC-Motoren: Vier bürstenlose Gleichstrommotoren wandeln die ESC-Signale in Schubkraft um.

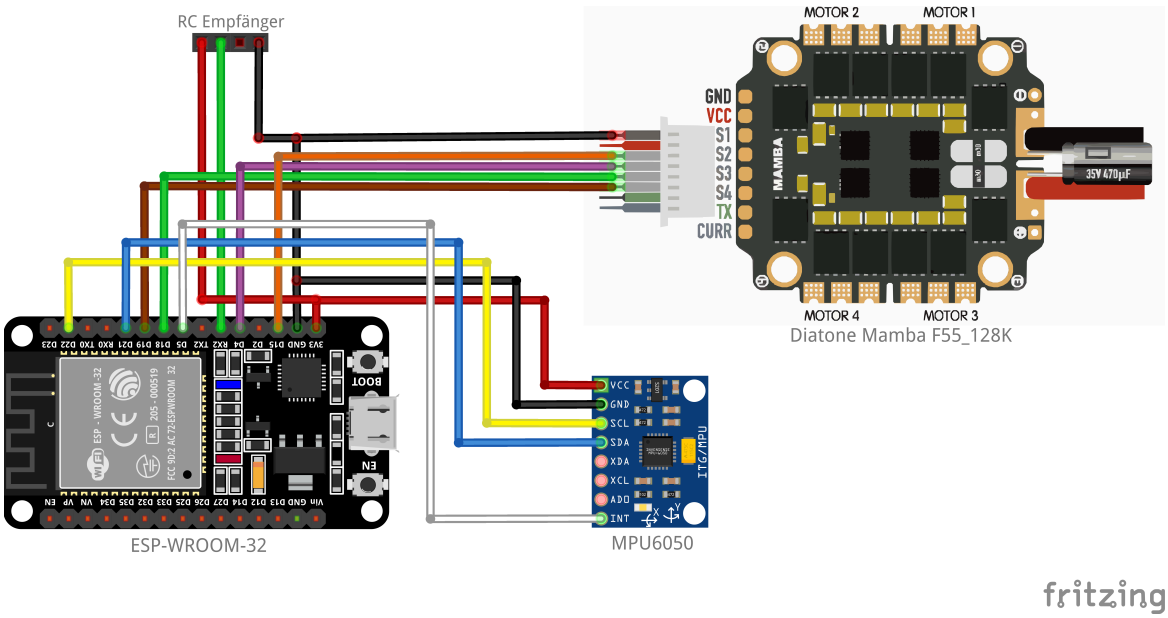


Abb. 2.3: Avionik des Quadrocoptersystems

Die Kommunikation mit der Bodenstation erfolgt derzeit kabelgebunden über eine serielle Schnittstelle. Eine drahtlose Anbindung ist jedoch leicht realisierbar, da das System bereits über einen Bluetooth-Adapter verfügt. Dieser muss lediglich an den UART-Port 0 angeschlossen werden, über den aktuell bereits die kabelgebundene Verbindung läuft. Somit müsste nur die Software der Bodenstation auf Bluetooth umgestellt werden.

2.2. Softwarearchitektur

2.2.1. Bodenstation Software

Die Bodenstation ermöglicht die Steuerung der Drohne sowie die Erfassung, Speicherung und Auswertung von Telemetriedaten. Hierfür wurde eine Desktop-Anwendung in Python unter Verwendung des GUI-Toolkits Tkinter entwickelt (vgl. Abb. 2.4). Die Software bietet eine modulare Konfiguration zur Echtzeitvisualisierung ausgewählter Telemetrieparameter. Darüber hinaus lassen sich verschiedene Trajektorien initiieren und automatisiert aufzeichnen, um sie im Anschluss anhand definierter Bewertungsgrößen wie z. B. einem Mean Squared Error zu analysieren. Zudem verarbeitet diese im Hintergrund die Positionsdaten des Lasertrackers und sendet diese über die serielle Kabelverbindung an den Mikrocontroller.

Kommt noch, wenn sie wirklich in der finalen Version ist

Abb. 2.4: GUI der Bodenstationssoftware

2.2.2. Onboard-Software

Für die Implementierung der Onboard-Software wurde das ESP-IDF Framework von Expressif Systems [5] verwendet, welches die grundlegenden Werkzeuge zur Entwicklung von Mikrocontroller Software liefern. Darunter fallen Softwarebibliotheken z. B. zur Verwendung von Hardwareperipherien aber auch Tools zur grundsätzlichen Kompilierung, zum Flashen und zum Monitorn des Mikrocontrollers. Da das Framework auf dem Echtzeitbetriebssystem freeRTOS basiert, erlaubt es effizientes Multitasking und trotzdem deterministisches Verhalten.

In der implementierten Onboard-Software wurden vier parallel laufende Tasks verwendet (Abb. 2.5), welche mit unterschiedlichen Taktraten laufen und für verschiedene Aufgaben zuständig sind:

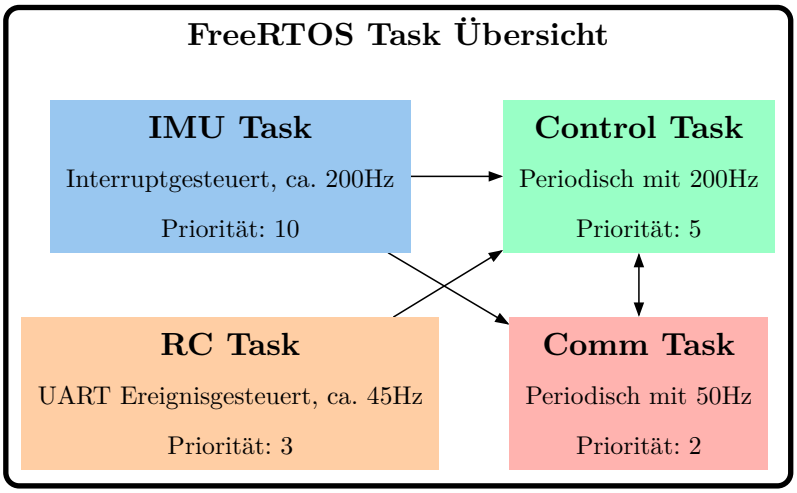


Abb. 2.5: FreeRTOS Tasks mit zugehörigen Datenflüssen

Der IMU-Task verarbeitet die Orientierungsdaten, die er von dem MPU6050 Sensor erhält. Er wird durch einen Hardware Interrupt durch den MPU6050 Sensor aktiviert, ließt die Daten aus der DMP-Warteschlange des Sensors aus und speichert die umgewandelten Roll-, Pitch- und Yaw-Werte ab, sodass diese vom Control- und Comm-Task abgefragt werden können. Der Sensor liefert gefilterte Lagedaten mit einer Taktrate von ca. 200Hz. Aufgrund der begrenzten Puffergröße des Sensors ist dieser Task mit der höchsten Priorität versehen, um fehlerhafte Daten zu vermeiden.

Wenn die Fernsteuerung aktiv ist, wertet der RC-Task ca. alle 22ms die Fernsteuerungsdaten aus und stellt diese an den Control-Task bereit. Dieser Task ist ereignisgesteuert und läuft, sobald neue RC-Daten auf dem UART Anschluss erhalten werden.

Der Control-Task arbeitet periodisch mit 200Hz und ist für die geregelte Ansteuerung der Aktoren zuständig. Die Lagedaten erhält er vom IMU-Task, die Positionsdaten vom Comm-Task. Je nach verwendetem Betriebsmodus liefert die Führungsgrößen entweder der RC-Task oder der Comm-Task. Erhält der Control-Task länger als 100ms keine neuen Mess- oder Führungsgrößen, so werden die Rotoren automatisch abgeschalten. Zusätzlich kann der Comm-Task auch konfiguriert werden, dass er verschiedene Parameter für den Regler liefern kann, wodurch sich diese dann online direkt von der Bodenstationssoftware aus konfigurieren lassen, ohne dass der Mikrocontroller-Code neu kompiliert und geflasht werden muss.

Der Comm-Task wertet alle 20ms von der Bodenstation erhaltene Befehle aus, konkret Führungsgrößen und Reglerparameter und sendet anschließend Telemetrie-Daten zurück. Diese enthalten Mess-, Führungs- und Stellgrößen sowie flexible Reglerparameter, welche unter anderem für Debugging-Zwecke genutzt werden können.

3. Modellbildung

Zur modellbasierten Reglerentwicklung sowie zur Simulation des Flugverhaltens wurde ein mathematisches Modell des Quadrocopters entworfen. Wie in Abb. 3.6 dargestellt lässt sich dieses in drei aufeinanderfolgende Teilmodelle unterteilen. Diese Struktur basiert auf der Annahme, dass die Antriebsdynamik nicht vom Zustand des Quadrocopters beeinflusst wird und das rotatorische Verhalten vollständig vom translatorischen Verhalten entkoppelt ist.

Dies stellt eine Vereinfachung der Realität dar, da sehr wohl physikalische Kopplungen zwischen diesen Teilen des Systems bestehen. Beispielsweise erhöht sich der Schub in Bodennähe (sog. Bodeneffekt, vgl. [6]) und während einer schnellen Vorwärtsbewegung des Copters verringert sich die Effizienz der Antriebe. Auf die mathematische Abbildung dieser komplexen Wechselwirkungen wird im vorliegenden Modell jedoch bewusst verzichtet, da deren Einfluss im moderaten, nicht hochdynamischen Flugbetrieb gering ist. [7]

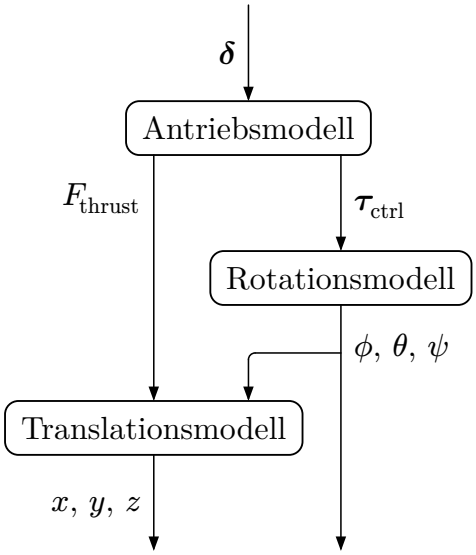


Abb. 3.6: Modellübersicht

Zur mathematischen Beschreibung des Systems werden das Inertialkoordinatenystem K_E und das körperfeste Koordinatensystem K_B mit Ursprung im Masseschwerpunkt des Quadrocopters eingeführt. Der Flugkörper wird in Plus-Konfiguration angenommen, was bedeutet, dass die Vorwärtsbewegung entlang der x-Achse parallel zu einem der Rotorarme verläuft (siehe Abb. 3.7). Das Inertialsystem K_E ist zu Beginn des Fluges deckungsgleich mit K_B und verbleibt ortsfest an dieser Ausgangsposition.

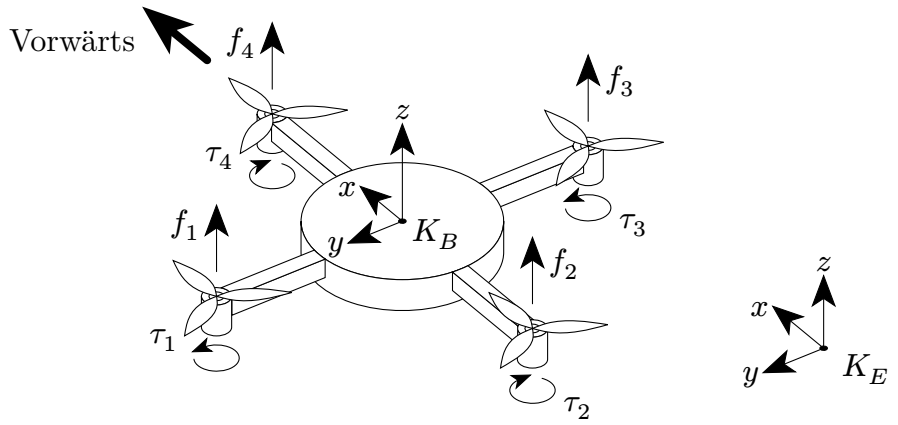


Abb. 3.7: Koordinatensysteme und Nummerierung der Rotoren

Zusätzlich werden die folgenden Größen definiert:

- im inertialen Raum K_E:

$$\text{Position } \boldsymbol{p} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \text{ Orientierung } \boldsymbol{\eta} = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix} \tag{3.1}$$

- in körperfesten Raum K_B:

$$\text{Winkelgeschwindigkeit } \boldsymbol{\omega} = \begin{bmatrix} p \\ q \\ r \end{bmatrix} \tag{3.2}$$

Zur Transformation von Vektoren mit Koordinaten in K_B zu Vektoren mit Koordinaten in K_E kann die Rotationsmatrix $\boldsymbol{R}$ in Abhängigkeit von der aktuellen Lage $\boldsymbol{\eta}$ des Quadrocopters verwendet werden [8]:

$$\boldsymbol{R}(\boldsymbol{\eta}) = \begin{bmatrix} \cos(\theta)\cos(\psi) & \sin(\phi)\sin(\theta)\cos(\psi) - \cos(\phi)\sin(\psi) & \cos(\phi)\sin(\theta)\cos(\psi) + \sin(\phi)\sin(\psi) \\ \cos(\theta)\sin(\psi) & \sin(\phi)\sin(\theta)\sin(\psi) + \cos(\phi)\cos(\psi) & \cos(\phi)\sin(\theta)\sin(\psi) - \sin(\phi)\cos(\psi) \\ -\sin(\theta) & \sin(\phi)\cos(\theta) & \cos(\phi)\cos(\theta) \end{bmatrix} \tag{3.3}$$

Antriebsmodell

Das Antriebsmodell beschreibt den Zusammenhang zwischen der Ansteuerung der Motoren und den resultierenden Kräften und Momenten auf den gesamten Quadrocopter. Die detaillierte Herleitung der folgenden Beziehungen kann [9] entnommen werden.

Vernachlässigt man dynamische Effekte, lassen sich die Schubkraft f_i und das Drehmoment τ_i eines einzelnen Rotors in Abhängigkeit der Drosselung $\delta_i \in [0, 1]$ wie folgt annähern:

$$f_i = a(\delta_i - b)^2 \tag{3.4}$$

$$\tau_i = c \cdot f_i \tag{3.5}$$

Die Parameter a, b und c repräsentieren keine direkten physikalischen Konstanten, sondern sind empirische Koeffizienten, die experimentell identifiziert werden müssen.

Mithilfe der Steuerungseffektivitätsmatrix $\boldsymbol{B}_{\text{eff}}$ werden die Einzelkräfte der Rotoren in den Vektor der verallgemeinerten Eingangsgrößen $\boldsymbol{u}$ transformiert. Dieser umfasst den Gesamtschub F_{thrust} sowie die Steuermomente $\boldsymbol{\tau}_{\text{ctrl}}$ um die drei körperfesten Achsen:

$$\boldsymbol{u} = \begin{bmatrix} F_{\text{thrust}} \\ \tau_{\text{ctrl,x}} \\ \tau_{\text{ctrl,y}} \\ \tau_{\text{ctrl,z}} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ l & 0 & -l & 0 \\ 0 & l & 0 & -l \\ c & -c & c & -c \end{bmatrix}}_{\boldsymbol{B}_{\text{eff}}} \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix} \tag{3.6}$$

Diese Matrix ist abhängig von der geometrischen Anordnung der Rotoren und deren Hebelarm l zum Schwerpunkt. Durch die gezielte Steuerung der vier Motordrosselungen können diese vier verallgemeinerten Eingangsgrößen unabhängig voneinander beeinflusst werden. Diese Eigenschaft ist essenziell für die Steuerallokation (siehe Abschnitt 4.1). Gleichzeitig lässt sich hier aber auch die Unteraktuierung des Systems erkennen, da trotzdem nur vier der sechs Freiheitsgrade unabhängig voneinander kontrolliert werden können.

Rotationsmodell

Für die Beschreibung der Rotationsdynamik wird die Euler'sche Gleichung verwendet, welche das Systemverhalten im körperfesten Koordinatensystem beschreibt, da sich hier der Trägheitstensor nicht verändert [8]:

$$\boldsymbol{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\boldsymbol{J}\boldsymbol{\omega}) = \boldsymbol{\tau} \tag{3.7}$$

Die Umrechnung der Winkelgeschwindigkeiten vom körperfesten Koordinatensystem in die zeitliche Ableitung der Euler-Winkel im Inertialsystem erfolgt über die Transformationsmatrix $\boldsymbol{T}$:

$$\dot{\boldsymbol{\eta}} = \boldsymbol{T}(\boldsymbol{\eta})\boldsymbol{\omega} \tag{3.8}$$

mit

$$\boldsymbol{T}(\boldsymbol{\eta}) := \begin{bmatrix} 1 & \sin(\phi)\tan(\theta) & \cos(\phi)\tan(\theta) \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \frac{\sin(\phi)}{\cos(\theta)} & \frac{\cos(\phi)}{\cos(\theta)} \end{bmatrix} \tag{3.9}$$

Diese Transformation unterscheidet sich von einer reinen Orientierungstransformation mit der Rotationsmatrix $\boldsymbol{R}$, da die einzelnen Euler-Winkelgeschwindigkeiten auf verschiedenen Zwischenkoordinatensystemen basieren.

Im freien Flug wird angenommen, dass nur die Steuermomente $\boldsymbol{\tau}_{\text{ctrl}}$ ein äußeres Drehmoment auf den Quadrocopter erzeugen:

$$\boldsymbol{\tau} = \boldsymbol{\tau}_{\text{ctrl}} \tag{3.10}$$

Bei dem Betrieb auf dem 3-DOF-Teststand entsteht aufgrund des vertikalen Versatzes h des Schwerpunkts zum Drehpunkt ein Gravitationsmoment $\boldsymbol{\tau}_g$. Die Drohne verhält sich hier wie ein instabiles, inverses Pendel. Es gilt:

$$\boldsymbol{\tau}_{\text{g}} = \begin{bmatrix} 0 \\ 0 \\ h \end{bmatrix} \times \boldsymbol{R}^{-1}(\boldsymbol{\eta}) \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} = \begin{bmatrix} mg\sin(\phi)\cos(\theta)h \\ mg\sin(\theta)h \\ 0 \end{bmatrix} \tag{3.11}$$

Zusätzlich soll auch noch ein dämpfendes Moment aufgrund von Reibung model liert werden, welches linear bzgl. der Winkelgeschwindigkeit angenommen wird:

$$\boldsymbol{\tau}_{\mathrm{d}} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} \omega^{\top} \tag{3.12}$$

Für das Gesamtmoment auf dem Teststand ergibt sich damit:

$$\boldsymbol{\tau} = \boldsymbol{\tau}_{\mathrm{ctrl}} + \boldsymbol{\tau}_{\mathrm{g}} + \boldsymbol{\tau}_{\mathrm{d}} \tag{3.13}$$

Translationsmodell

Das Translationsmodell wird vollständig im Inertialkoordinatensystem entwickelt. Die Bewegungsgleichungen basieren auf dem 2. Newton'schen Axiom:

$$m\ddot{\boldsymbol{p}} = \boldsymbol{F} \tag{3.14}$$

Dabei wird angenommen, dass die Gesamtkraft $\boldsymbol{F}$ ausschließlich aus der Gewichtskraft und der Schubkraft besteht:

- Gewichtskraft $\boldsymbol{F}_g$: wirkt in negativer z-Richtung des Inertialsystems

$$\boldsymbol{F}_{\mathrm{g}} = \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} \tag{3.15}$$

- Schubkraft $\boldsymbol{F}_{\mathrm{thrust}}$: wirkt in positiver z-Richtung des körperfesten Koordinatensystem. Mithilfe der Rotationsmatrix $\boldsymbol{R}$ (vgl. Gleichung (3.3)) wird die Kraft in das Inertialsystem transformiert:

$$\boldsymbol{F}_{\mathrm{thrust}} = \boldsymbol{R}(\boldsymbol{\eta}) \begin{bmatrix} 0 \\ 0 \\ F_{\mathrm{thrust}} \end{bmatrix} \tag{3.16}$$

Die resultierende Gesamtkraft ergibt sich somit zu:

$$\boldsymbol{F} = \boldsymbol{F}_{\mathrm{g}} + \boldsymbol{F}_{\mathrm{thrust}} \tag{3.17}$$

Gesamtmodell im Zustandraum

Diese Modelle lassen sich gemeinsam im Zustandsraum beschreiben. Der Zustandsvektor $\boldsymbol{x}$ umfasst die Position $\boldsymbol{p}$, die Geschwindigkeit $\dot{\boldsymbol{p}}$, die Orientierung $\boldsymbol{\eta}$ und die Winkelgeschwindigkeit $\boldsymbol{\omega}$:

$$\boldsymbol{x} = [\boldsymbol{p}, \dot{\boldsymbol{p}}, \boldsymbol{\eta}, \boldsymbol{\omega}]^\top \tag{3.18}$$

Die Systemdynamik $\dot{\boldsymbol{x}}$ sowie die Ausgangsgleichung $\boldsymbol{y}$ ergeben sich damit zu:

$$\dot{\boldsymbol{x}} = \begin{bmatrix} \dot{\boldsymbol{p}} \\ \ddot{\boldsymbol{p}} \\ \dot{\boldsymbol{\eta}} \\ \dot{\boldsymbol{\omega}} \end{bmatrix} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}) = \begin{bmatrix} \boldsymbol{f}_t(\boldsymbol{x}, \boldsymbol{u}) \\ \boldsymbol{f}_r(\boldsymbol{x}, \boldsymbol{u}) \end{bmatrix} = \begin{bmatrix} \dot{\boldsymbol{p}} \\ m^{-1}\boldsymbol{F} \\ \boldsymbol{T}\boldsymbol{\omega} \\ \boldsymbol{J}^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega} \times (\boldsymbol{J}\boldsymbol{\omega})) \end{bmatrix} \tag{3.19}$$

$$\boldsymbol{y} = [\boldsymbol{p}, \boldsymbol{\eta}]^\top$$

3.1. Parameteridentifikation

Wie in [9] dargelegt, lassen sich nahezu alle Systemparameter durch gezielte Versuchsaufbauten unabhängig vom Flugbetrieb bestimmen. Die auf diese Weise identifizierten Parameter sind in Tabelle 3.1 zusammengefasst.

Parameter	Wert
a	13.06 N
b	0.0859
c	0.0351 m
l	0.2 m
m	1.210 kg
h	0.05 m
$\boldsymbol{J}$	$\begin{bmatrix} 0.0258 & 0 & 0 \\ 0 & 0.0268 & 0 \\ 0 & 0 & 0.0680 \end{bmatrix}$ kg m^2

Tabelle 3.1: Nicht im Flug identifizierte Systemparameter

Mit diesen ist bereits eine hinreichend genaue Simulation möglich, um erste stabilisierende Regler für das reale System zu entwerfen. Trotzdem lässt sich der Sim-to-Real-Gap durch Erweiterung des Modells und der Verbesserung bereits identifizierter Parameter weiter schließen.

Da jedoch die Identifikation mit spezifischen Testständen recht zeit- und hardwareintensiv ist, wurde in Julia ein Verfahren zur Identifikation im geschlossenen Regelkreis implementiert. Dieses erlaubt es, Parameter direkt aus Daten des regu-

lären Flugbetriebs zu extrahieren, was den Identifikationsprozess maßgeblich beschleunigt.

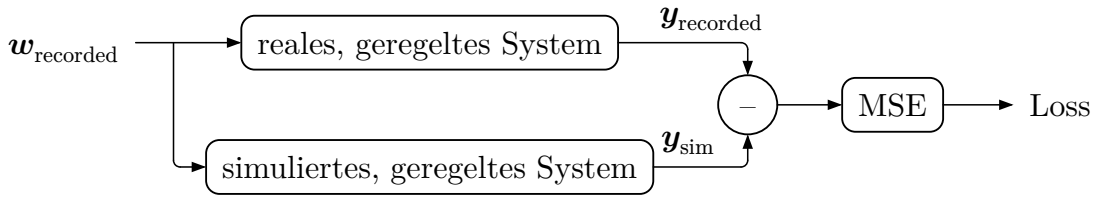


Abb. 3.8: Optimierungsfunktion

Der Identifikationsprozess gliedert sich hierbei in folgende Schritte:

1. Aufzeichnung von Führungs- und Ausgangsgrößen direkt am realen Flugsystem
2. Definition der Optimierungsfunktion $f_{\text{loss}}(\boldsymbol{\omega}_{\text{recorded}}, \boldsymbol{y}_{\text{recorded}})$ (Abb. 3.8), welche die Abweichung zwischen Realität und Modell angibt.
3. Durchführung einer gradientenbasierten Optimierung mittels BFGS-Verfahren. Da Julia automatische Differentiation unterstützt, kann der Gradient effizient über die gesamte Simulationsstrecke hinweg bestimmt werden. Wichtig: Die Reglerparameter in Simulation und Realität müssen identisch sein.

Bei der Identifikation im geschlossenen Regelkreis ergeben sich im Vergleich zum offenen Regelkreis zwei zentrale Herausforderungen [10]:

1. Störungen auf die Ausgangsgröße sind direkt mit den Eingangsgrößen des Systems korreliert. Dadurch kann das simulierte Systemverhalten trotz guter Parameter auf Dauer stark von dem realen Verhalten abweichen.
2. Ein Regler ist meist darauf ausgelegt, einen Großteil der Eigendynamik zu unterdrücken. Dadurch kann es schwerer sein, die Parameter der echten Streckendynamik aus den aufgenommen Daten zu extrahieren.

Deshalb ist eine ausreichende Anregung der Systemdynamik entscheidend, damit sich diese auch in den aufgenommen Daten wiederspiegelt und mögliche Störungen überdeckt. Deshalb wurde der Regler absichtlich mit verschiedenen, nicht optimalen Parametern betrieben und es wurden verschiedenste Trajektorien abgeflogen, um eine möglichst breite Anregung des gesamten Dynamikbereichs zu gewährleisten.

Mithilfe dieses Verfahrens konnten die Dämpfungskonstanten p_x, p_y und p_z das erste Mal identifiziert werden. Ebenso wurden die Parameter h und $\boldsymbol{J}$ neu ermittelt, da deren bisherige Bestimmung nur eine begrenzte Genauigkeit zuließ. Die bereits vorhanden Werte dienten dabei als Startwerte für den Optimierungsalgorithmus. Um die Plausibilität der Ergebnisse zu gewährleisten, wurde der zulässige Suchraum auf eine Abweichung von maximal 20% begrenzt.

Die so identifizierten Parameter finden sich in Tabelle 3.2:

Parameter	Wert
p_x	0.051 Nm s rad^{-1}
p_y	0.040 Nm s rad^{-1}
p_z	0.142 Nm s rad^{-1}
h	0.0421 m
$\boldsymbol{J}$	$\begin{bmatrix} 0.0276 & 0 & 0 \\ 0 & 0.0295 & 0 \\ 0 & 0 & 0.0551 \end{bmatrix}$ kg m^2

Tabelle 3.2: Im Flug identifizierte Systemparameter

Ein Vergleich der Simulation vor und nach der Identifikation (siehe Abb. 3.9) be legt, dass die Optimierung den Sim-to-Real-Gap im Regelkreis effektiv minimiert hat.

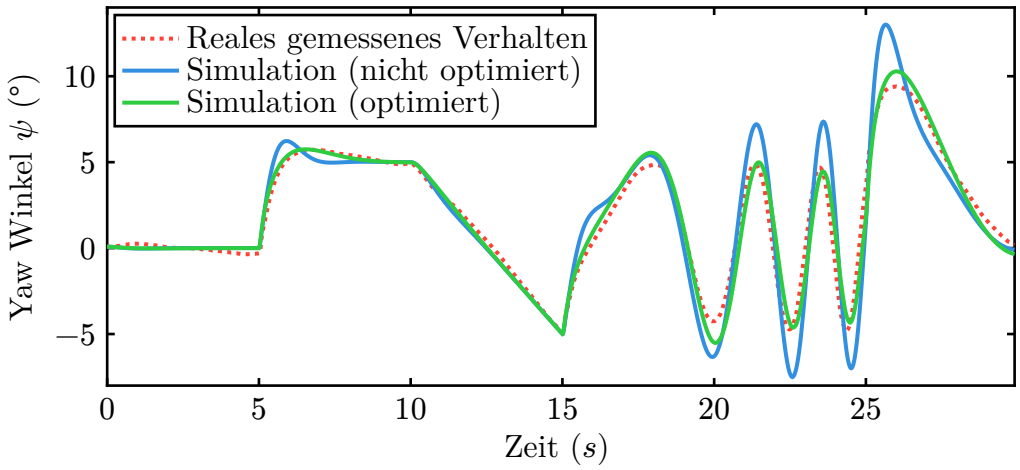


Abb. 3.9: Simulation des Yaw-Verhaltens vor und nach der Optimierung

3.2. Linearisiertes Modell

Ein lineares Zustandsraummodell ist die Grundvoraussetzung für viele modellbasierte Regelungsverfahren. In dieser Arbeit bildet es die Basis für den Entwurf der beiden LQ-Regler, kann jedoch gleichermaßen für komplexere Ansätze wie die modellprädiktive Regelung genutzt werden. Um für die Zukunft eine maximale Flexibilität für den Einsatz des Modells zu gewährleisten, soll das zuvor hergeleitete nichtlineare Systemmodell zunächst allgemein ohne festen Arbeitspunkt linearisiert werden. Erst im Anschluss wird es dann noch speziell um die Ruhelage der Hoverposition linearisiert.

Die Antriebsdynamik wurde als statisches System modelliert, wodurch sie nicht Teil des Zustandsraumes ist und deshalb nicht linearisiert werden muss.

Rotationsdynamik

Die Linearisierung der Rotationsdynamik $\dot{\boldsymbol{x}}_\mathrm{r} = \boldsymbol{f}_\mathrm{r}(\boldsymbol{x}_\mathrm{r}, \boldsymbol{\tau}_\mathrm{ctrl})$ erfolgt um einen frei wählbaren Arbeitspunkt $\boldsymbol{x}_{\mathbf{r,0}} = [\boldsymbol{\eta}_\mathbf{0}, \boldsymbol{\omega}_\mathbf{0}]^\top$ und das zugehörige Eingangsmoment $\boldsymbol{\tau}_{\mathbf{ctrl,0}}$. Es stellt sich wie folgt auf:

$$
\begin{aligned}
\Delta \boldsymbol{x}_\mathrm{r} &= \boldsymbol{x}_\mathrm{r} - \boldsymbol{x}_{\mathrm{r},0} \\
\Delta \boldsymbol{\tau}_\mathrm{ctrl} &= \boldsymbol{\tau}_\mathrm{ctrl} - \boldsymbol{\tau}_{\mathrm{ctrl},0} \\
\Delta \dot{\boldsymbol{x}}_\mathrm{r} &= \boldsymbol{f}_\mathrm{r}\big(\boldsymbol{x}_{\mathrm{r},0}, \boldsymbol{\tau}_{\mathrm{ctrl},0}\big) + \boldsymbol{A}_\mathrm{r} \Delta \boldsymbol{x}_\mathrm{r} + \boldsymbol{B}_\mathrm{r} \Delta \boldsymbol{\tau}_\mathrm{ctrl} \\
\boldsymbol{y}_\mathrm{r} &= \boldsymbol{C}_\mathrm{r} \boldsymbol{x}_{\mathrm{r},0} + \boldsymbol{C}_\mathrm{r} \Delta \boldsymbol{x}_\mathrm{r}
\end{aligned}
\tag{3.20}
$$

Für die Systemmatrix A_r ergeben sich zwei Varianten, abhängig davon, ob der Quadrocopter im Freiflug oder auf dem Teststand betrieben wird:

$$
\boldsymbol{A}_{\mathrm{r,\ frei}}\big(\boldsymbol{x}_{\boldsymbol{r},\mathbf{0}}\big) = \begin{bmatrix}
\big(\omega_y \cos(\phi) - \omega_z \sin(\phi)\big) \tan(\theta) & \frac{\omega_y \sin(\phi) + \omega_z \cos(\phi)}{\cos^2(\theta)} & 0 & 1 & \sin(\phi)\tan(\theta) & \cos(\phi)\tan(\theta) \\
-\omega_y \sin(\phi) - \omega_z \cos(\phi) & 0 & 0 & 0 & \cos(\phi) & -\sin(\phi) \\
\frac{\omega_y \cos(\phi) - \omega_z \sin(\phi)}{\cos(\theta)} & \frac{(\omega_y \sin(\phi) + \omega_z \cos(\phi)) \sin(\theta)}{\cos^2(\theta)} & 0 & 0 & \frac{\sin(\phi)}{\cos(\theta)} & \frac{\cos(\phi)}{\cos(\theta)} \\
0 & 0 & 0 & 0 & \frac{\omega_z(J_{yy} - J_{zz})}{J_{xx}} & \frac{\omega_y(J_{yy} - J_{zz})}{J_{xx}} \\
0 & 0 & 0 & \frac{\omega_z(-J_{xx} + J_{zz})}{J_{yy}} & 0 & \frac{\omega_x(-J_{xx} + J_{zz})}{J_{yy}} \\
0 & 0 & 0 & \frac{\omega_y(J_{xx} - J_{yy})}{J_{zz}} & \frac{\omega_x(J_{xx} - J_{yy})}{J_{zz}} & 0
\end{bmatrix}
\tag{3.21}
$$

$$\boldsymbol{A}_{\mathrm{r,\ test}}\left(\boldsymbol{x}_{\boldsymbol{r},\mathbf{0}}\right) = \begin{bmatrix} \left(\omega_y \cos(\phi) - \omega_z \sin(\phi)\right) \tan(\theta) & \frac{\omega_y \sin(\phi) + \omega_z \cos(\phi)}{\cos^2(\theta)} & 0 & 1 & \sin(\phi)\tan(\theta) & \cos(\phi)\tan(\theta) \\ -\omega_y \sin(\phi) - \omega_z \cos(\phi) & 0 & 0 & 0 & \cos(\phi) & -\sin(\phi) \\ \frac{\omega_y \cos(\phi) - \omega_z \sin(\phi)}{\cos(\theta)} & \frac{\left(\omega_y \sin(\phi) + \omega_z \cos(\phi)\right)\sin(\theta)}{\cos^2(\theta)} & 0 & 0 & \frac{\sin(\phi)}{\cos(\theta)} & \frac{\cos(\phi)}{\cos(\theta)} \\ \frac{ghm\cos(\phi)\cos(\theta)}{J_{xx}} & -\frac{ghm\sin(\phi)\sin(\theta)}{J_{xx}} & 0 & -\frac{d_x}{J_{xx}} & \frac{\omega_z(J_{yy}-J_{zz})}{J_{xx}} & \frac{\omega_y(J_{yy}-J_{zz})}{J_{xx}} \\ 0 & \frac{ghm\cos(\theta)}{J_{yy}} & 0 & \frac{\omega_z(-J_{xx}+J_{zz})}{J_{yy}} & -\frac{d_y}{J_{yy}} & \frac{\omega_x(-J_{xx}+J_{zz})}{J_{yy}} \\ 0 & 0 & 0 & \frac{\omega_y(J_{xx}-J_{yy})}{J_{zz}} & \frac{\omega_x(J_{xx}-J_{yy})}{J_{zz}} & -\frac{d_z}{J_{zz}} \end{bmatrix} \tag{3.22}$$

Im Gegensatz zur Systemmatrix sind die Matrizen $\boldsymbol{B}_{\mathrm{r}}$ und $\boldsymbol{C}_{\mathrm{r}}$ unabhängig vom gewählten Arbeitspunkt, da die zugrunde liegenden Gleichungen bereits einen li nearen Zusammenhang aufweisen:

$$\boldsymbol{B}_{\mathbf{r}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ \frac{1}{J_{xx}} & 0 & 0 \\ 0 & \frac{1}{J_{yy}} & 0 \\ 0 & 0 & \frac{1}{J_{zz}} \end{bmatrix}, \quad \boldsymbol{C}_{\mathbf{r}} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \tag{3.23}$$

In der Praxis erfolgt die Regelung meist um eine stationäre Ruhelage. Bei einem Quadrocopter befindet sich diese bei $\boldsymbol{x}_{\mathrm{r,\ AP}} = [0, 0, \varphi_0, 0, 0, 0]^\top$. Im Folgenden soll zusätzlich $\psi_0 = 0$ angenommen werden. Für diesen speziellen Arbeitspunkt erge ben sich die beiden Varianten für die Systemmatrix zu:

$$\boldsymbol{A}_{r,\ \mathrm{frei}}(\boldsymbol{x}_{\mathrm{AP}}) = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{3.24}$$

$$\boldsymbol{A}_{r,\ \mathrm{test}}(\boldsymbol{x}_{\mathrm{AP}}) = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ \frac{mgh}{J_{\mathrm{xx}}} & 0 & 0 & -\frac{p_x}{J_{\mathrm{xx}}} & 0 & 0 \\ 0 & \frac{mgh}{J_{\mathrm{yy}}} & 0 & 0 & -\frac{p_y}{J_{\mathrm{yy}}} & 0 \\ 0 & 0 & 0 & 0 & 0 & -\frac{p_z}{J_{\mathrm{zz}}} \end{bmatrix} \tag{3.25}$$

Translationdynamik

Analog zur Rotation wird die Translationsdynamik $\dot{\boldsymbol{x}}_\mathrm{t} = \boldsymbol{f}_\mathrm{t}(\boldsymbol{x}_\mathrm{t})$ um den Zustand $\boldsymbol{x}_{\mathrm{t},0} = [\boldsymbol{p}_0, \dot{\boldsymbol{p}}_0]^\top$ und den Eingang $\boldsymbol{u}_{\mathrm{t},0} = \left[F_{\mathrm{thrust},0}, \phi_0, \theta_0, \psi_0\right]$ linearisiert.

$$\begin{aligned}
\Delta\boldsymbol{x}_\mathrm{t} &= \boldsymbol{x}_\mathrm{t} - \boldsymbol{x}_{\mathrm{t},0} \\
\Delta\boldsymbol{u}_\mathrm{t} &= \boldsymbol{u}_\mathrm{t} - \boldsymbol{u}_{\mathrm{t},0} \\
\Delta\dot{\boldsymbol{x}}_\mathbf{t} &= \boldsymbol{f}_\mathbf{t}\big(\boldsymbol{x}_{\mathrm{t},0}, \boldsymbol{u}_{\mathrm{t},0}\big) + \boldsymbol{A}_\mathrm{t}\Delta\boldsymbol{x}_\mathrm{t} + \boldsymbol{B}_\mathrm{t}\Delta\boldsymbol{u}_\mathrm{t} \\
\boldsymbol{y} &= \boldsymbol{C}_\mathrm{t}\boldsymbol{x}_{\mathrm{t},0} + \boldsymbol{C}_\mathrm{t}\Delta\boldsymbol{x}_\mathrm{t}
\end{aligned} \tag{3.26}$$

Hierbei weist lediglich die Eingangsmatrix $\boldsymbol{B}_\mathrm{t}$ eine Abhängigkeit vom gewählten Arbeitspunkt auf, während die Systemmatrix $\boldsymbol{A}_\mathrm{t}$ und die Ausgangsmatrix $\boldsymbol{C}_\mathrm{t}$ konstant bleiben.

$$\boldsymbol{A}_\mathrm{t} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad \boldsymbol{C}_\mathrm{t} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \tag{3.27}$$

$$\boldsymbol{B}_\mathbf{t}\big(\boldsymbol{u}_{\mathrm{t},\,0}\big) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \frac{\sin(\phi)\sin(\psi)+\sin(\theta)\cos(\phi)\cos(\psi)}{m} & \frac{F_{thrust}(-\sin(\phi)\sin(\theta)\cos(\psi)+\sin(\psi)\cos(\phi))}{m} & \frac{F_{thrust}\cos(\phi)\cos(\psi)\cos(\theta)}{m} & \frac{F_{thrust}(\sin(\phi)\cos(\psi)-\sin(\psi)\sin(\theta)\cos(\phi))}{m} \\ \frac{-\sin(\phi)\cos(\psi)+\sin(\psi)\sin(\theta)\cos(\phi)}{m} & -\frac{F_{thrust}(\sin(\phi)\sin(\psi)\sin(\theta)+\cos(\phi)\cos(\psi))}{m} & \frac{F_{thrust}\sin(\psi)\cos(\phi)\cos(\theta)}{m} & \frac{F_{thrust}(\sin(\phi)\sin(\psi)+\sin(\theta)\cos(\phi)\cos(\psi))}{m} \\ \frac{\cos(\phi)\cos(\theta)}{m} & -\frac{F_{thrust}\sin(\phi)\cos(\theta)}{m} & -\frac{F_{thrust}\sin(\theta)\cos(\phi)}{m} & 0 \end{bmatrix} \tag{3.28}$$

Für den speziellen Fall der Hover-Position, in der die Auftriebskraft die Gewichtskraft exakt kompensiert und alle Neigungswinkel Null sind, ergibt sich für die Eingangsmatrix:

$$\boldsymbol{B}_\mathrm{t}(\boldsymbol{u}_\mathrm{AP}) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & g & 0 \\ 0 & -g & 0 & 0 \\ \frac{1}{m} & 0 & 0 & 0 \end{bmatrix} \tag{3.29}$$

4. Reglerentwurf

Quadrocopter sind inhärent instabile Systeme, da keine natürlichen Rückstellkräfte existieren, die äußeren Störungen entgegenwirken. Beispielsweise führt ein störendes Drehmoment um die Roll-Achse zu einer bleibenden Änderung der Winkelgeschwindigkeit, welche ohne aktiven Eingriff nur sehr langsam durch aerodynamischen Widerstand abgebaut würde. Während sich die Winkelgeschwindigkeit im freien Flug somit näherungsweise grenzstabil verhält, ist der Rollwinkel durch die Integration der Winkelgeschwindigkeit bereits instabil. [7] Auf dem verwendeten Teststand verschärft sich dieses Verhalten noch weiter, da hier sogar schon die Winkelgeschwindigkeit aufgrund des versetzten Schwerpunktes instabil ist. Analog dazu zeigt auch das translatorische Verhalten Instabilitäten.

Dieses Systemverhalten macht eine aktive Regelung unumgänglich. Sie dient aber nicht nur der reinen Stabilisierung sondern sorgt auch dafür, dass das System Trajektorien möglichst dynamisch folgen kann.

Die implementierte Regelungsstruktur besteht aus drei Komponenten: die Positionsregelung, die Lageregelung und die Steuerungszuweisung (siehe Abb. 4.10). Wie auch schon in der Modellbildung wurde die Regelung des Systems in rotatorisches und translatorisches Verhalten unterteilt. Diese Trennung ist in der Praxis aus folgenden Gründen als Standard etabliert [11]:

1. Verringerung der Komplexität: Die Struktur erlaubt es, den Lageregler noch unabhängig von der Positionsregelung zu entwickeln. Sobald dieser dann die gewünschte Dynamik vorweist, kann darauf aufbauend der Positionsregler entwickelt werden.
2. Möglichkeit der Kaskadenreglung: Das rotatorische Verhalten ist deutlich dynamischer als das translatorische. Zusätzlich können Lagedaten mit höherer Fre-

quenz gemessen werden als die Positionsdaten. Daher ist es sinnvoll, die Lageregelung getrennt in einer inneren Schleife mit höherer Taktfrequenz auszuführen.

3. Erhöhte Flexibilität: Bei FPV-Drohnen ist beispielsweise im Flugcomputer nur eine Lageregelung implementiert. Die Positionsregelung übernimmt der Pilot. Dies ist mit dieser Aufteilung relativ leicht umsetzbar.

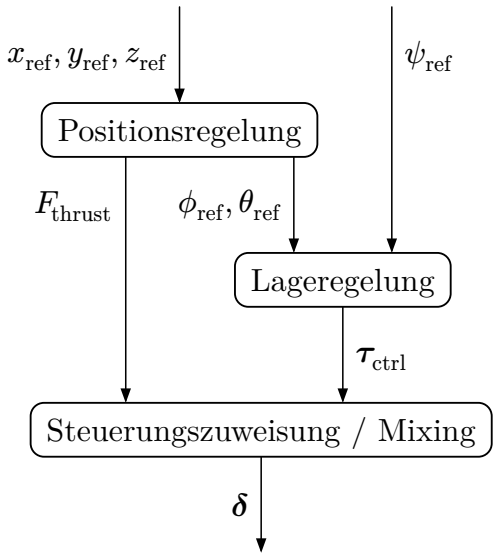


Abb. 4.10: Reglerübersicht

Als Führungsgrößen für den freien Flug dienen die Position $[x_{\text{ref}}, y_{\text{ref}}, z_{\text{ref}}]^\top$ sowie der Gierwinkel ψ_{ref} verwendet. Der Regler berechnet daraus den Stellgrößenvektor $\boldsymbol{\delta} = [\delta_1, \delta_2, \delta_3, \delta_4]^\top$, welcher die Ansteuerung der vier Motoren definiert.

Im Folgenden werden die drei Kernkomponenten der Regelungsstruktur detailliert erläutert. Für die Positions- sowie die Lageregelung werden jeweils zwei unterschiedliche Reglerentwürfe vorgestellt und vergleichend evaluiert. Abschließend wird die Vorgehensweise zur Implementierung der Algorithmen auf dem Zielsystem dargelegt.

4.1. Steuerungszuweisung

Die Aufgabe der Steuerungszuweisung besteht darin, die von den Reglern geforderten Kräfte und Momente in die Stellbefehle für die einzelnen Motoren umzuwandeln. Im Kontext von Multicoptern wird dies auch häufig als Mixing bezeichnet.

Da sowohl die Steuerungseffektivitätsmatrix $\boldsymbol{B}_{\text{eff}}$ (vgl. Gleichung (3.6)) als auch der Antriebszusammenhang (vgl. Gleichung (3.4)) invertierbar sind, lassen sich die Stellgrößen direkt berechnen. Es gilt:

$$\begin{bmatrix} f_{1,\text{ ref}} \\ f_{2,\text{ ref}} \\ f_{3,\text{ ref}} \\ f_{4,\text{ ref}} \end{bmatrix} = \boldsymbol{B}_{\text{eff}}^{-1} \begin{bmatrix} F_{\text{thrust}} \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} \tag{4.30}$$

$$\delta_{\text{i, ref}} = \sqrt{\frac{f_{\text{i, ref}}}{a}} + b \tag{4.31}$$

Komplexität entsteht erst, sobald die Stellgrößenbegrenzung berücksichtigt werden muss. Der Arbeitsbereich eines Motors liegt bei einer Drosselung zwischen 15 % und 90 %. Eine einfache Limitierung der einzelnen Motordrehzahlen ist jedoch problematisch: Tritt ein Motor in die Sättigung, ändert sich nicht nur die Amplitude der Schubkraft, sondern insbesondere die Richtung des Momentvektors. Dies führt zu Kopplungen in andere Freiheitsgrade, die vom Regler nicht vorgesehen waren und damit die Stabilität gefährden können.

Um dieses Problem zu lösen, müssen bereits die verallgemeinerten Stellgrößen $\boldsymbol{u}$ begrenzt werden. Dies ist mathematisch anspruchsvoll, da die Begrenzung nicht für jede Komponente isoliert, sondern in Abhängigkeit der Gesamtsumme und unter Beibehaltung der Richtung des Drehmomentvektors erfolgen muss. Zusätzlich wird meist der Lagestabilität eine höhere Bedeutung im Vergleich zum Erhalt der Höhe beigemessen, was ebenfalls bei der Steuerungszuweisung beachtet werden muss. [11]

Wird der Quadrocopter jedoch mit einer Schubkraft von ca. 50 % und mit nicht sehr hoher Dynamik betrieben, so werden die Sättungsbereiche selten erreicht. Da dies bei dem untersuchten System als Vorraussetzung fesgesetzt werden kann, wird in dieser Arbeit die Motorstellgrößen trotz der genannten Problematik direkt limitiert. Sollten dadurch kurzzeitige Sättigungseffekte auftreten, so sind die Regler meist robust genug, um die dadurch entstehenden Störmomente relativ schnell auszugleichen.

4.2. Lageregelung

Wie bereits zu Beginn des Kapitels erläutert wurde, ist das Rotationsverhalten inhärent instabil und muss stabilisiert werden. Da das System nahe der Ruhelage betrieben werden soll, liefert das in Abschnitt 3.2 entwickelte linearisierte Rotationsmodell hier eine gute Approximation. Aus diesem geht hervor, dass die einzelnen Achsen in der Nähe der Ruhelage als entkoppelt betrachtet werden können. Deshalb ist es möglich, die Regler für die einzelnen Achsen getrennt voneinander zu entwickeln.

Das Übertragungsverhalten einer einzelnen Achse entspricht einem System 2. Ordnung, bei dem die Stellgröße direkt auf die Beschleunigung des Orientierungswinkels wirkt. Um einen geeigneten Regler zu entwerfen, wird das linearisierte Verhalten am Beispiel der Roll-Achse auf dem Teststand genauer betrachtet. Für dieses gilt:

$$\ddot{\phi} = \frac{1}{J_{\mathrm{xx}}}\left(-p_x\dot{\phi} + mgh\phi + \tau_x\right) \tag{4.32}$$

Unter Verwendung des allgemeinen Regelgesetzes

$$\tau_x = -k_p\phi - k_d\dot{\phi} \tag{4.33}$$

lässt sich das System als homogene Differentialgleichung zweiter Ordnung darstellen und kann in die folgende Form überführt werden:

$$\ddot{y} = -a_1\dot{y} - a_0 y \tag{4.34}$$

Solche idealen Systeme sind asymptotisch stabil, wenn $a_0, a_1 > 0$ [11]. Diese Bedingung ließe sich bereits durch einen reinen P-Regler erfüllen. In diesem Fall wäre die Dämpfung jedoch allein durch aerodynamische Effekte gegeben und somit sehr gering, was zu einer hohen Schwingung des Systems führen wird. Zusätzlich vernachlässigt das idealisierte Modell zusätzliche verzögernde Faktoren, die das reale System trotzdem destabilisieren würden, wie z. B. Aktorverzögerungen, Abtastzeiten des diskreten Reglers und Totzeiten in der Sensordatenverabeitung.

Durch die Hinzunahme eines D-Anteils kann die Phasenreserve erhöht und die Systemantwort gedämpft werden, wodurch auch das reale System zuverlässig stabilisiert wird.

Das System besitzt im Betrieb auf dem Teststand kein integrierendes Verhalten, da $a_0 \neq 0$ aufgrund des versetzten Schwerpunkts gilt. Deshalb benötigt dieses zwingend einen I-Anteil, um stationäre Genauigkeit zu erreichen. In freifliegender Konfiguration könnte hingegen theoretisch auf den I-Anteil verzichtet werden, da hier $a_0 = 0$ gilt und das System damit bereits ein integrierendes Verhalten aufweist. Jedoch ist auch hier ein I-Anteil meist sinnvoll, um die stationäre Genauigkeit zu verbessern. Bei der Verwendung eines I-Anteils muss beachtet werden, dass dieser eine zusätzliche Verzögerung in das System einführt, was die Phasenreserve verringert und damit die Stabilität gefährden kann. Deshalb muss die Verstärkung des I-Anteils sorgfältig gewählt werden, um einen guten Kompromiss zwischen Regelgüte und Stabilität zu erreichen. [12]

4.2.1. PID

Die einfachste Methode, um einen Regler mit solchen Eigenschaften umzusetzen ist der PID-Regler (Abb. 4.11). Dieser wendet die proportionalen, integralen und derivativen Anteile direkt auf die Regelabweichung $e = \varphi_{\text{ref}} - \varphi_{\text{meas}}$ an, wodurch automatisch eine Folgeregelung realisiert wird. [12] Um die Verstärkung von hochfrequenten Messrauschen zu begrenzen, wird ein realer D-Anteil mit einem Tiefpassfilter $G(s) = \frac{N}{s+N}$ verwendet. Dieser verhindert extrem hohe Amplituden in der Stellgröße, welche das System destabilisieren können.

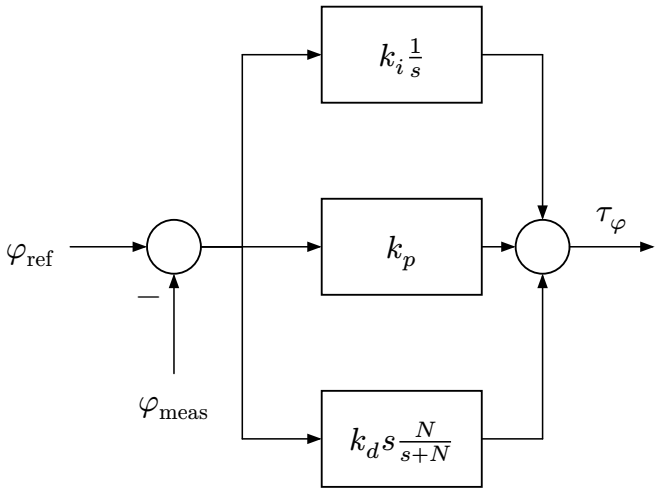


Abb. 4.11: PID-Regelung einer Achse

Die Parameter des PID-Reglers wurden zunächst in Simulation empirisch ermittelt und anschließend noch am realen System auf dem Teststand weiter optimiert. Die Ergebnisse sind in der folgenden Tabelle dargestellt:

	k_p	k_i	k_d	N
Roll	1.40	1.18	0.35	100
Pitch	1.42	1.20	0.38	100
Yaw	1.20	0.70	0.50	100

Tabelle 4.3: Empirisch ermittelte PID-Parameter für Lageregelung

Es ist zu erkennen, dass die Parameter für die Roll- und Nick-Achse relativ ähnlich sind. Dies lässt sich auf die strukturelle Symmetrie des Systems zurückführen. Bei der Gier-Achse fällt der P-Anteil trotz des höheren Trägheitsmomentes geringer aus, da hier kein destabilisierendes Moment kompensiert werden muss. Der geringere P-Anteil für die Gier-Achse trotz höherem Trägheitsmoment lässt sich damit erklären, dass es kein destabilisierendes Moment gibt, welches ausgeglichen werden muss. Auch der I-Anteil ist niedriger dimensioniert, da die Gier-Achse bereits inhärent ein integrierendes Verhalten aufweist. Dieser dient hier nur für eine Verstärkung dieses Verhaltens. Der D-Anteil ist hingegen höher, was sich mit der höheren Trägheit um diese Achse erklären lässt.

4.2.2. LQR mit integraler Ausgangsrückführung

Im Gegensatz zum klassischen PID-Regler basiert der Linear-Quadratische Regler (LQR) auf einer Zustandsrückführung anstelle einer reinen Ausgangsrückführung. Da im vorliegenden System lediglich die Euler-Winkel direkt gemessen werden können, müssen die zur vollständigen Zustandsbeschreibung notwendigen Winkelgeschwindigkeiten geschätzt werden. Hierzu wird ein Kalman-Filter eingesetzt [13]. Dieser wägt die Messgüte, also das Sensorrauschen, gegen die Modellgüte, also die Modellgenauigkeit, optimal ab, um eine statistisch optimale Zustandsschätzung zu liefern. Um die Komplexität gering zu halten, wird hierbei ein vereinfachtes Integratormodell anstelle der vollständigen Systemdynamik verwendet.

Um ein präzises Führungsverhalten sowie stationäre Genauigkeit zu gewährleisten, wird das System um einen Vorfilter $\boldsymbol{S}$ und eine integrale Ausgangsrückführung erweitert. Daraus ergibt sich die folgende Systemstruktur:

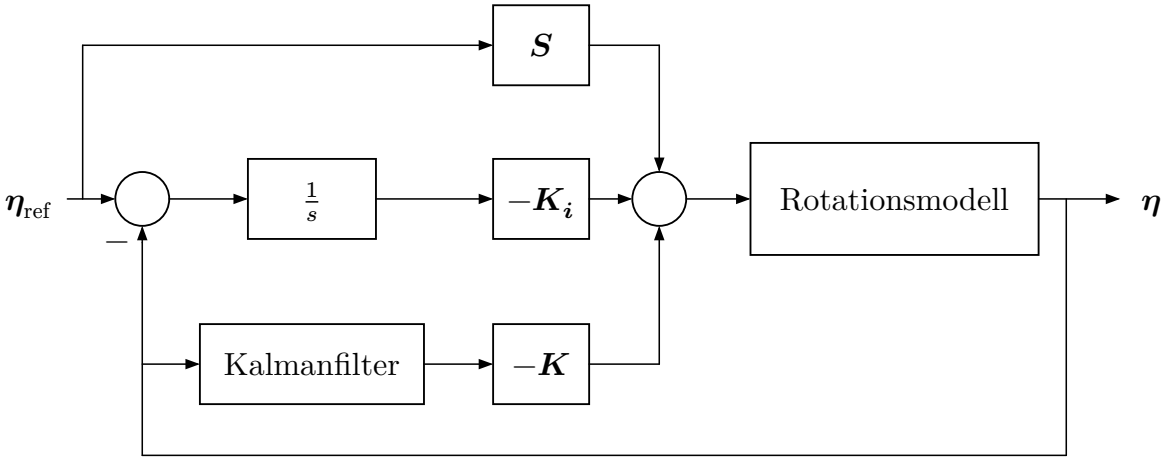


Abb. 4.12: Struktur des Zustandreglers mit integraler Ausgangsrückführung und Vorfilter

Da in dem System die einzelnen Achsen als entkoppelt betrachtet werden können, weist der LQR mit integraler Rückführung strukturell große Ähnlichkeiten zu drei separaten PID-Reglern auf. Der einzige Unterschied liegt in der Umsetzung des D-Anteils. Während dieser beim PID-Regler durch eine gefilterte numerische Differenzierung des Regelfehlers realisiert wird, nutzt der LQR die vom Kalman-Filter

geschätzte Winkelgeschwindigkeit. Diese geht direkt in das Stellgesetz ein und das ohne die Ableitung der Führungsgröße.

Der entscheidende Vorteil ist deshalb wenig struktureller Natur, sondern resultiert primär aus der Methodik der Parameterbestimmung. Während der PID-Regler eine manuelle Abstimmung erfordert, erfolgt diese beim LQR durch eine mathematische Optimierung auf Basis des Systemmodells. Wenn die Modellgüte hinreichend genau ist wird beim LQR zusätzlich Stabilität garantiert.

Zur Bestimmung der Reglermatrizen $\boldsymbol{K}$ und $\boldsymbol{K}_i$ wird das Zustandsraummodell um die integrierten Fehler der Euler-Winkel $\boldsymbol{x}_{\mathrm{I,\,r}}$ erweitert. Das resultierende erweiterte System lautet:

$$\frac{d}{dt}\begin{bmatrix}\boldsymbol{x}_{\mathrm{r}}\\ \boldsymbol{x}_{\mathrm{I,\,r}}\end{bmatrix} = \begin{bmatrix}\boldsymbol{A}_{\mathrm{r}} & \boldsymbol{0}\\ -\boldsymbol{C}_{\mathrm{r}} & \boldsymbol{0}\end{bmatrix}\begin{bmatrix}\boldsymbol{x}_{\mathrm{r}}\\ \boldsymbol{x}_{\mathrm{I,\,r}}\end{bmatrix} + \begin{bmatrix}\boldsymbol{B}\\ \boldsymbol{0}\end{bmatrix}\boldsymbol{\tau}_{\mathbf{ctrl}} + \begin{bmatrix}\boldsymbol{0}\\ \boldsymbol{I}\end{bmatrix}\boldsymbol{\eta}_{\mathbf{ref}} \tag{4.35}$$

Für dieses System sind die Gewichtungsmatrizen $\boldsymbol{Q}$ und $\boldsymbol{R}$ der Kostenfunktion so zu wählen, dass ein optimaler Kompromiss zwischen Regelgüte und Stellaufwand erzielt wird. In [14] wird dabei Bryson's Regel für die initiale Wahl der Gewichtungsmatrizen angewandt. Diese besagt, dass die Matrizen als Diagonalmatrizen gewählt werden sollen, deren Einträge dem Kehrwert des Quadrats der maximal zulässigen Abweichungen entsprechen:

$$\boldsymbol{Q} = \mathrm{diag}\left(\frac{1}{\max\left(\boldsymbol{x}_{\mathrm{r,\,erw}}\right)^2}\right), \quad \boldsymbol{R} = \mathrm{diag}\left(\frac{1}{\max\left(\boldsymbol{\tau}_{\mathrm{ctrl}}\right)^2}\right) \tag{4.36}$$

Als Grenzwerte wurden für die Euler-Winkel sowie deren integrierte Fehler jeweils 1° und für die Winkelgeschwindigkeit 5°/s definiert. Das Limit für die Stellgrößen wurde zunächst auf 0,03 Nm festgesetzt, da dies in etwa dem maximalen Moment bei einer Drosselung von 50 % entspricht, die den Motor nicht in den Sättigungsbereich treibt.

In Testläufen zeigte sich jedoch, dass die Stellgröße bei dieser Konfiguration dennoch häufig die Sättigungsgrenzen erreichte. Um den Linearitätsbereich der Motoren zu wahren, wurde die Gewichtung der Stellgröße letztlich um den Faktor 2,5 erhöht. Daraus ergeben sich gerundet die folgenden Gewichtungsmatrizen:

$$\boldsymbol{Q} = \mathrm{diag}(3300, 3300, 3300, 130, 130, 130, 3300, 3300, 3300) \tag{4.37}$$

$$\boldsymbol{R} = \mathrm{diag}(2500, 2500, 2500) \tag{4.38}$$

Da die Implementierung auf einem digitalen System erfolgt, muss das Optimierungsproblem für das diskretisierte Systemmodel $(\boldsymbol{A}_\mathrm{d}, \boldsymbol{B}_\mathrm{d})$ gelöst werden. Hierzu wird zunächst die diskrete algebraische Riccati-Gleichung nach $\boldsymbol{P}$ aufgelöst:

$$\boldsymbol{P} = \boldsymbol{A}_\mathrm{d}{}^\top \boldsymbol{P} \boldsymbol{A}_\mathrm{d} - \left(\boldsymbol{A}_\mathrm{d}{}^\top \boldsymbol{P} \boldsymbol{B}_\mathrm{d}\right)\left(\boldsymbol{R} + \boldsymbol{B}_\mathrm{d}{}^T \boldsymbol{P} \boldsymbol{B}_\mathrm{d}\right)^{-1}\left(\boldsymbol{B}_\mathrm{d}{}^\top \boldsymbol{P} \boldsymbol{A}_\mathrm{d}\right) + \boldsymbol{Q} \tag{4.39}$$

Die optimale, erweiterte Reglermatrix $[\boldsymbol{K}, \boldsymbol{K}_i]$ ergibt sich anschließend zu:

$$[\boldsymbol{K}, \boldsymbol{K}_i] = \left(\boldsymbol{R} + \boldsymbol{B}_\mathrm{d}{}^\top \boldsymbol{P} \boldsymbol{B}_\mathrm{d}\right)^{-1} \boldsymbol{B}_\mathrm{d}{}^\top \boldsymbol{P} \boldsymbol{A}_\mathrm{d} \tag{4.40}$$

In MATLAB kann die Diskretierung des Systems und die Reglersynthese in einem Schritt mit dem Befehl `lqrd(A, B, Q, R, Ts)` durchgeführt werden. [15]

4.2.3. Evaluierung

Zur Evaluierung der Regelgüte wurden beide Algorithmen für jede Achse einzeln anhand einer einheitlichen Referenztrajektorie getestet. Diese Trajektorie umfasst Sprungantworten, Rampenfunktionen sowie harmonische Schwingungen, um ein breites Spektrum an Systemdynamiken abzudecken.

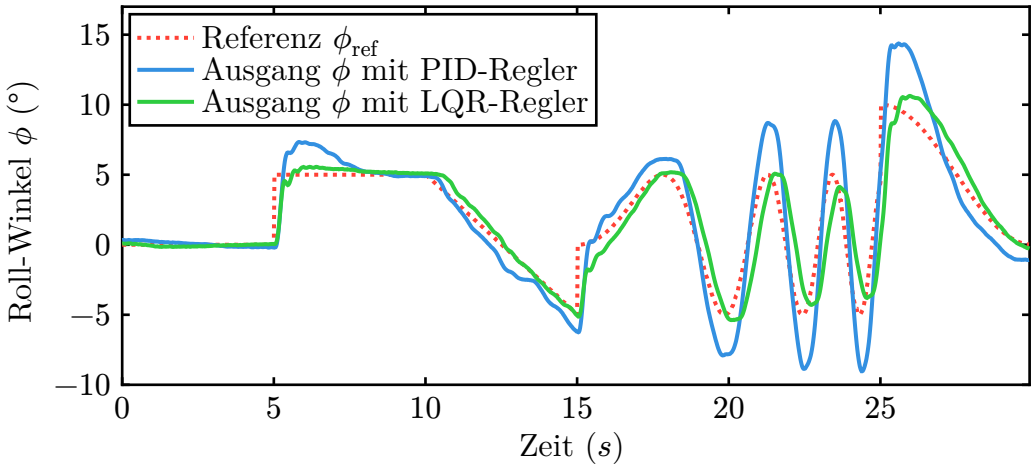
Um die Leistungsfähigkeit der Regler zu quantifizieren, wurde die Wurzel des mittleren quadratischen Fehlers zwischen der Referenz und dem gemessenen Ausgang für jede Achse berechnet. Die Ergebnisse sind in der folgenden Tabelle dargestellt:

Achse	PID	LQR
Roll	1.93°	1.71°
Nick	1.70°	1.72°
Gier	0.90°	1.75°

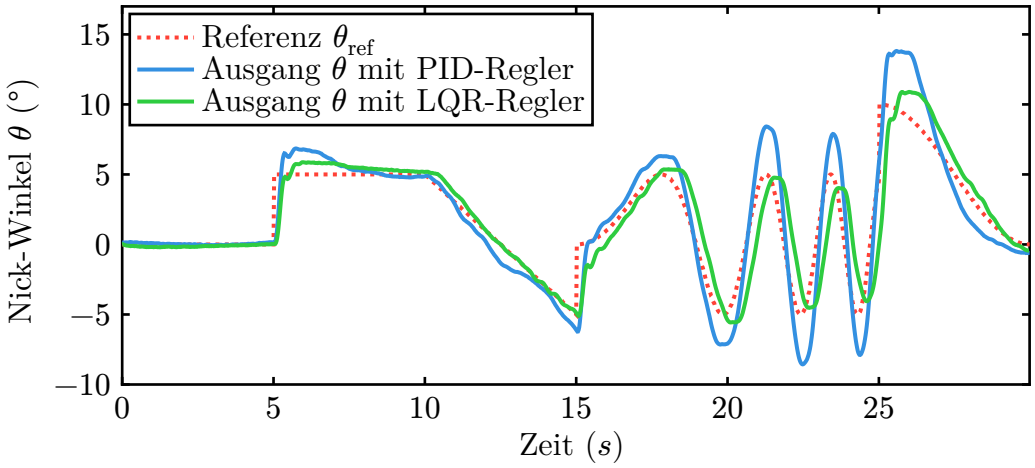
Tabelle 4.4: Vergleich der Regelgüte zwischen PID- und LQR-Regler anhand des Root Mean Squared Error

Wie bereits beim Reglerentwurf erläutert, weisen beide Strukturen eine hohe Ähnlichkeit auf. Ein wesentlicher Vorteil des LQR liegt jedoch in der Kombination mit dem Kalman-Filter. Da die Schätzung der Winkelgeschwindigkeit deutlich unempfindlicher gegenüber Messrauschen ist als die numerische Differenzierung des Fehlers mit Tiefpassfilter, erlaubt der LQR höhere D-Anteile, wodurch die Dynamik deutlich verbessert werden kann. Jedoch ließe sich auch der PID-Regler mit einem Kalman-Filter kombinieren, um die Vorteile der verbesserten Dämpfung zu nutzen. Somit ist dieser Vorteil nicht inhärent für die LQR-Struktur an sich.

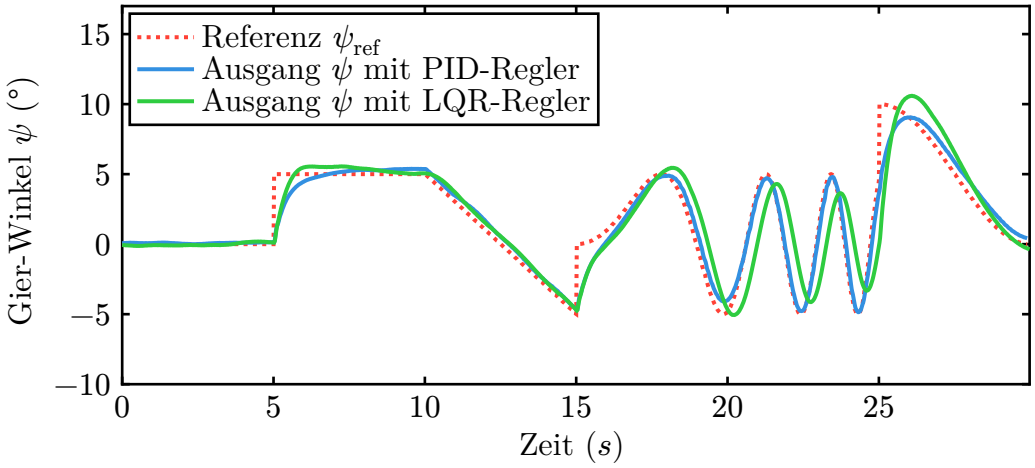
Im Gegensatz dazu erlaubt der PID-Regler eine intuitivere manuelle Feinabstimmung, da die Parameter hier nicht durch das zugrunde liegende Systemmodell gegenseitig beschränkt werden. Im Gesamtergebnis performen beide Ansätze bezüglich des Gütekriteriums ähnlich, wobei der LQR auf der Roll-Achse eine leicht höhere Regelgüte erzielt. Auf der Gier-Achse hingegen überzeugt der PID-Regler, während der LQR hier ein deutlich schwächeres Ergebnis liefert.



(a) Roll-Achse



(b) Nick-Achse



(c) Gier-Achse

Abb. 4.13: Verhalten mit PID-Regler

4.3. Positionsregelung

4.3.1. PID

4.3.2. LQR mit integraler Ausgangsrückführung

4.3.3. Evaluierung

4.4. Implementierung

5. Fazit und Ausblick

Literaturverzeichnis

[1] K. Nonami, „Research and Development of Drone and Roadmap to Evolution“, *Journal of Robotics and Mechatronics*, Bd. 30, Nr. 3, S. 322–336, 2018, doi: 10.20965/jrm.2018.p0322.

[2] „A Review of Modeling and Control Techniques for Unmanned Aerial Vehicles - Gedefaw - 2025 - Engineering Reports - Wiley Online Library“. Zugegriffen: 23. Oktober 2025. [Online]. Verfügbar unter: https://onlinelibrary.wiley.com/doi/full/10.1002/eng2.70215

[3] „Quadrotor Control System“. Zugegriffen: 3. März 2026. [Online]. Verfügbar unter: https://www.emqopter.de/de/quadrotor_control_system.php

[4] „MAMBA F55_128K BL32 4IN1 ESC 55A 6S“. Zugegriffen: 3. März 2026. [Online]. Verfügbar unter: https://www.diatone.us/products/mb-f55_128k-bl32-esc

[5] „Espressif/Esp-Idf“. Zugegriffen: 4. März 2026. [Online]. Verfügbar unter: https://github.com/espressif/esp-idf

[6] J.-W. Lee, N. Xuan-Mung, N. P. Nguyen, und S. K. Hong, „Adaptive Altitude Flight Control of Quadcopter under Ground Effect and Time-Varying Load: Theory and Experiments“, *Journal of Vibration and Control*, Bd. 29, Nr. 3–4, S. 571–581, Feb. 2023, doi: 10.1177/10775463211050169.

[7] J. Stephan, *Multicopter Flight Control: Fundamentals of Flight Mechanics, Control Allocation, and Control Design.* in Advances in Industrial Control. Cham: Springer Nature Switzerland, 2025. doi: 10.1007/978-3-031-78487-3.

[8] G. S. Krishnaswami, „Rigid Body Mechanics“, *Classical Mechanics: From Particles to Continua and Regularity to Chaos*. Springer Nature, Singapore, S. 271–320, 2025. doi: 10.1007/978-981-97-4476-3_8.

[9] „Modellbildung Und Identifikation Eines Quadrocopters“.

[10] P. Van den Hof, „Closed-Loop Issues in System Identification“, *Annual Reviews in Control*, Bd. 22, S. 173–186, Jan. 1998, doi: 10.1016/S1367-5788(98)00016-9.

[11] Q. Quan, *Introduction to Multicopter Design and Control*. Singapore: Springer, 2017. doi: 10.1007/978-981-10-3382-7.

[12] P. D. J. Lunze, *Regelungstechnik 1: Systemtheoretische Grundlagen, Analyse und Entwurf einschleifiger Regelungen*. Berlin, Heidelberg: Springer, 2020. doi: 10.1007/978-3-662-60746-6.

[13] J. Lunze, *Regelungstechnik 2*. Berlin, Heidelberg: Springer, 2016. doi: 10.1007/978-3-662-52676-7.

[14] E. Okyere, A. Bousbaine, G. T. Poyi, A. K. Joseph, und J. M. Andrade, „LQR Controller Design for Quad-Rotor Helicopters“, *The Journal of Engineering*, Bd. 2019, Nr. 17, S. 4003–4007, 2019, doi: 10.1049/joe.2018.8126.

[15] „lqrd - Design discrete linear-quadratic (LQ) regulator for continuous plant - MATLAB“. Zugegriffen: 18. März 2026. [Online]. Verfügbar unter: https://de.mathworks.com/help/control/ref/lqrd.html

Abbildungen

Abb. 2.1 Systemüberblick (Fernsteuerung ist optional) 4
Abb. 2.2 Quadrocopter auf dem 3-DOF-Teststand 5
Abb. 2.3 Avionik des Quadrocoptersystems 6
Abb. 2.4 GUI der Bodenstationssoftware 7
Abb. 2.5 FreeRTOS Tasks mit zugehörigen Datenflüssen 8
Abb. 3.6 Modellübersicht 10
Abb. 3.7 Koordinatensysteme und Nummerierung der Rotoren 11
Abb. 3.8 Optimierungsfunktion 16
Abb. 3.9 Simulation des Yaw-Verhaltens vor und nach der Optimierung 17
Abb. 4.10 Reglerübersicht 22
Abb. 4.11 PID-Regelung einer Achse 26
Abb. 4.12 Struktur des Zustandreglers mit integraler Ausgangsrückführung und Vorfilter 27
Abb. 4.13 Verhalten mit PID-Regler 31

Tabellen

Tabelle 3.1 Nicht im Flug identifizierte Systemparameter 15
Tabelle 3.2 Im Flug identifizierte Systemparameter 17
Tabelle 4.3 Empirisch ermittelte PID-Parameter für Lageregelung 26
Tabelle 4.4 Vergleich der Regelgüte zwischen PID- und LQR-Regler anhand des Root Mean Squared Error 29

Erklärung

Die vorliegende Arbeit habe ich selbstständig ohne Benutzung anderer als der angegebenen Quellen angefertigt. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten Quellen entnommen wurden, sind als solche kenntlich gemacht. Die Arbeit ist in gleicher oder ähnlicher Form oder auszugsweise im Rahmen einer oder anderer Prüfungen noch nicht vorgelegt worden.

Augsburg, den 20.03.2026 Maximilian Schweiger